

ImageLab

Asynchronous Image Processing

202 Accepted + Durable Job + One Worker + 1-Second Short Polling

Reynerio Samos · Chahiim Pop

CMPS 4121 · Assessment 1 · Version 1

Go · PostgreSQL · HTML/CSS/Vanilla JS · Docker

Agenda

- 01 What ImageLab does
- 02 Architecture — the mental model
- 03 API contract: 202 + durable job
- 04 Worker lifecycle & variants
- 05 Short polling & UI states
- 06 Error handling: retrieval vs processing
- 07 Measurements & the five-job burst
- 08 Acceptance checklist, demo, reflection

One central mental model

ImageLab separates **when a request is accepted** from **when the work is done**.



The POST accepts and preserves. The worker performs. The database records. The browser observes.

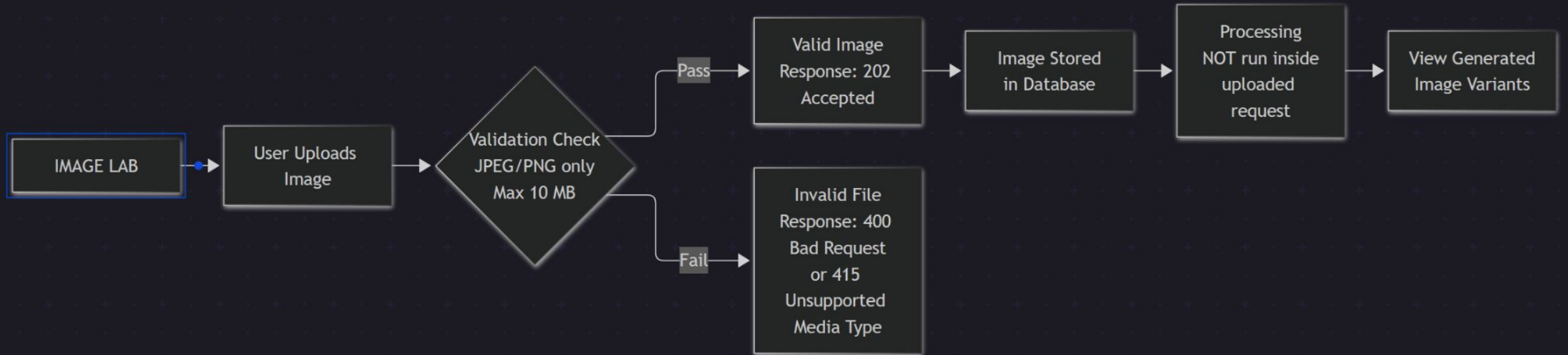
Browser to Variants

browser → go api → postgres → worker → storage

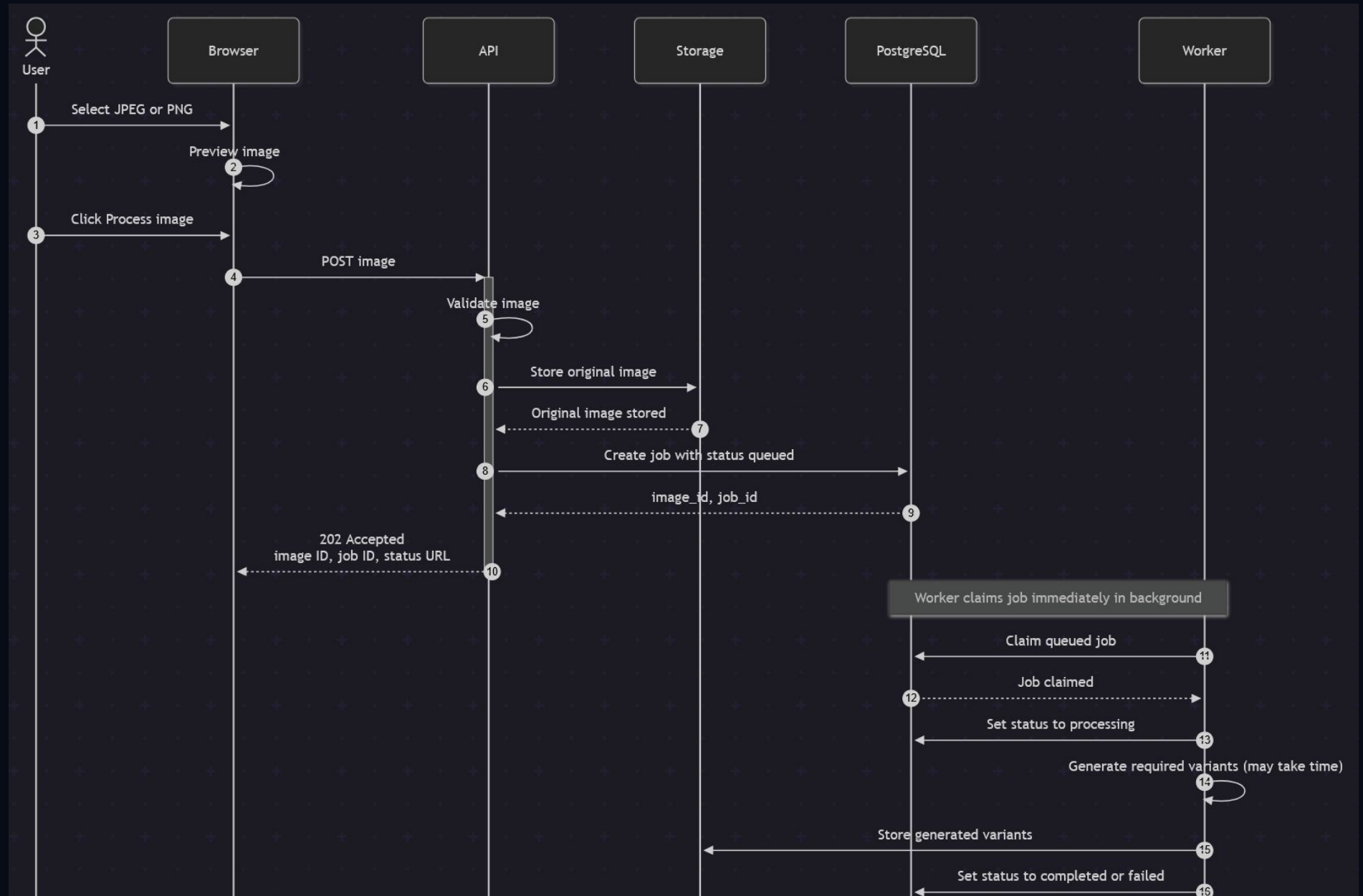


↓ display all variants stored by the worker in storage/variants

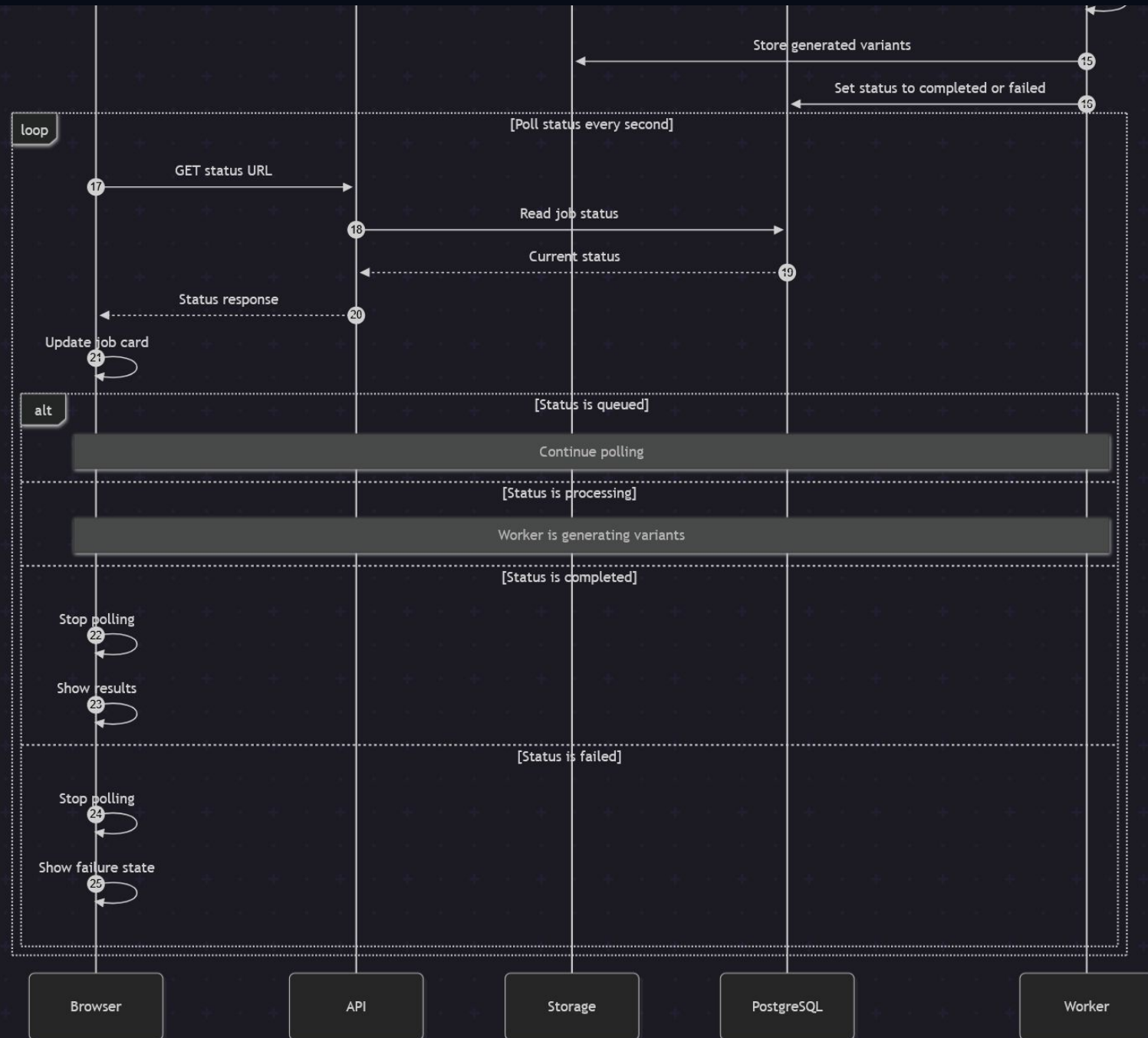
Contract Flow



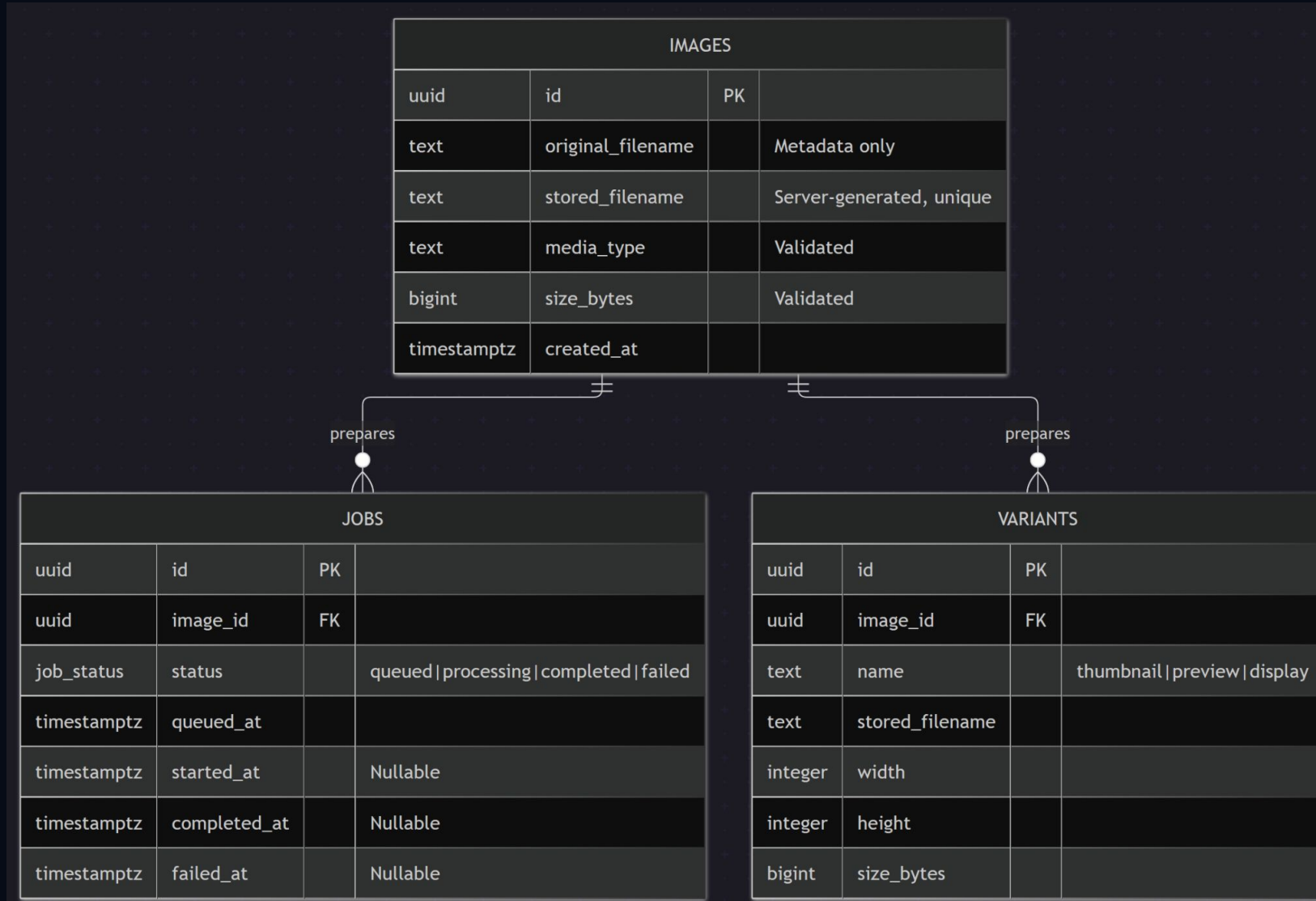
User Flow



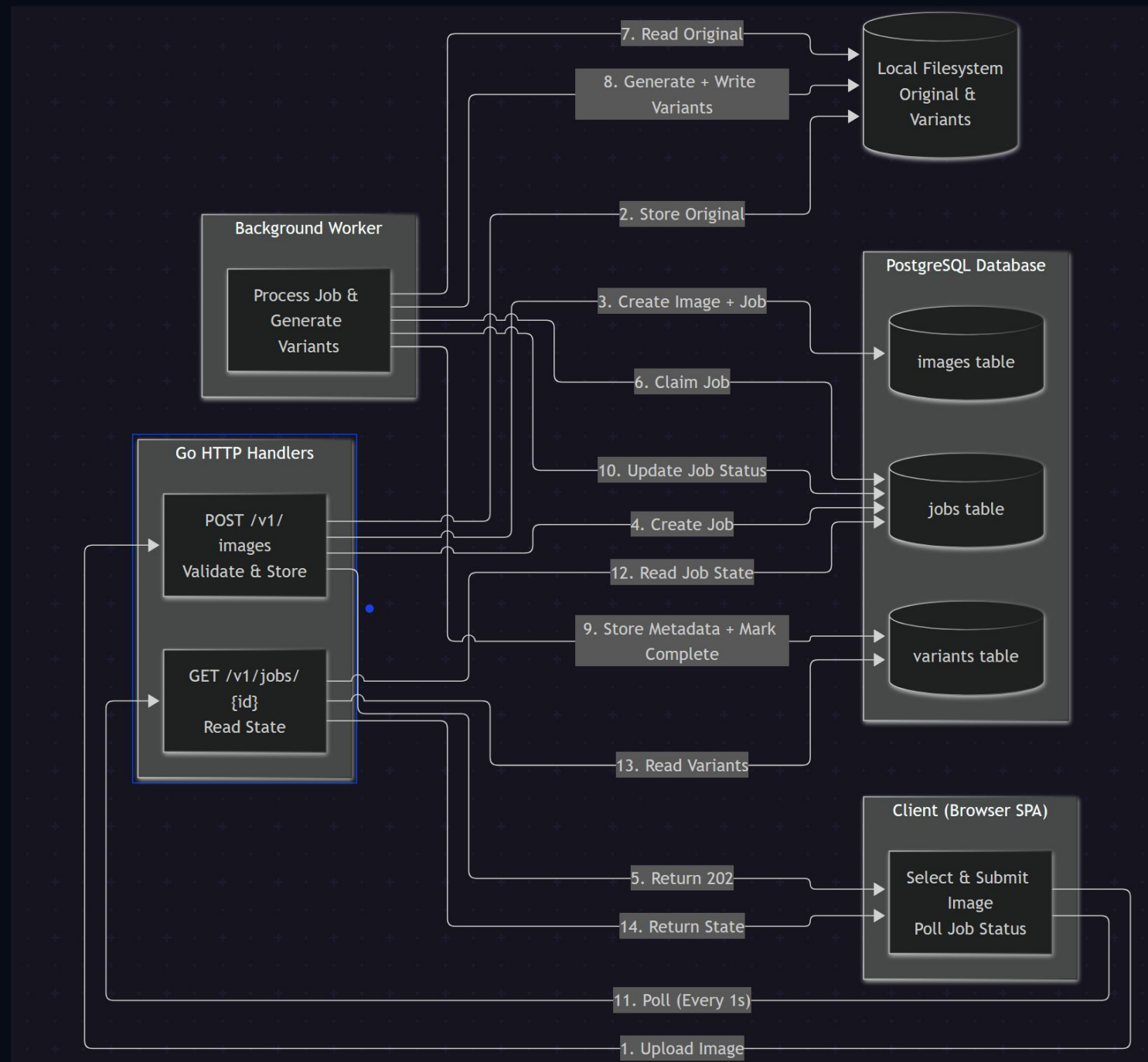
User Flow Cont.



Database Schema



Architecture Overview



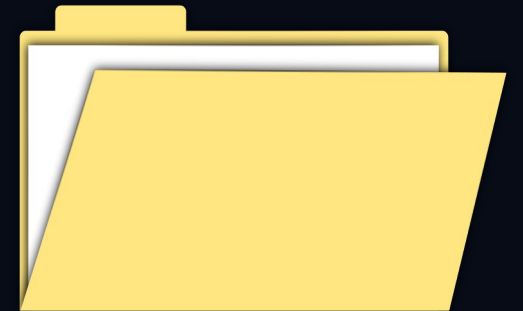
Component responsibilities

Component	Responsibility
Browser	Select & preview image, submit once, poll job state, render UI and results
Go API	Validate, store the original, create the durable job, return 202, expose resources
PostgreSQL	Owns authoritative image, job, state, timestamps, error, and variant metadata
Worker	Claims queued work, generates variants, persists results, marks completed or failed
Storage	Stores original and generated image bytes with server-controlled names

Required technologies

Required

Server	Go
Database	PostgreSQL
Client	HTML, CSS, Vanilla JavaScript
Storage	Local filesystem
Async	202 + persisted Job + one worker
Observation	Automatic short polling ≈ 1 sec



POST /v1/images → 202 Accepted

Response body

HTTP/1.1 202 Accepted

Location: /v1/jobs/{job_id}

```
{
  "image_id": "...",
  "job_id": "...",
  "status": "queued",
  "status_url": "/v1/jobs/{job_id}"
}
```

Guarantees

- Work is durably persisted before the response
- Response arrives before processing finishes
- Client can observe progress via status_url
- Original image stored, job created as queued

 ***Variants are never generated inside the upload handler.***

Why 202 Accepted? Acknowledge now, work later

202 decouples **request lifetime** from **background-work lifetime**.

1

Accept fast

Validate, store, insert the job — then return now.

2

Work async

The worker generates variants after the response.

3

State visible

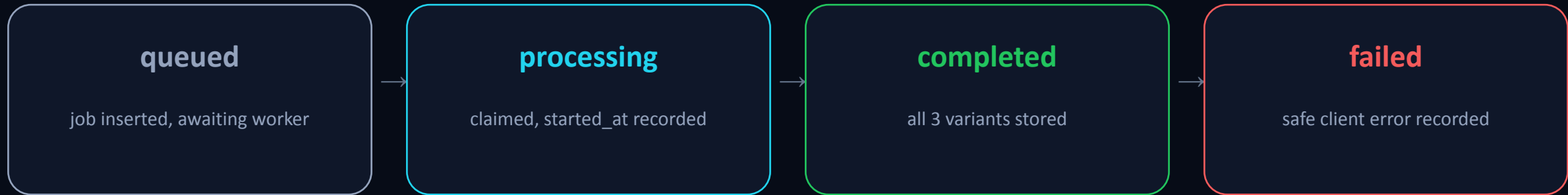
Each state transition is committed to PostgreSQL.

4

Observe later

Polling follows the job, not the original request.

Job states — one authoritative source



Timestamps for measurement

queued_at → started_at → completed_at / failed_at

Evidence: psql SELECT — single image end-to-end shows the full transition chain.

One worker, full lifecycle

- 1** **Every 500 ms** Ticker checks for queued jobs
- 2** **Claim** Picks next queued job, records started_at
- 3** **Process** Decodes original, generates 3 variants
- 4** **Persist** Writes variant files + metadata rows
- 5** **Transitions** MarkCompleted / MarkFailed, then next job

Runner model: one worker makes queueing visible when several jobs are submitted.

Three required variants

thumbnail

Fast, exact-square preview used at small sizes

preview

Fit within a bounded box, aspect preserved

display

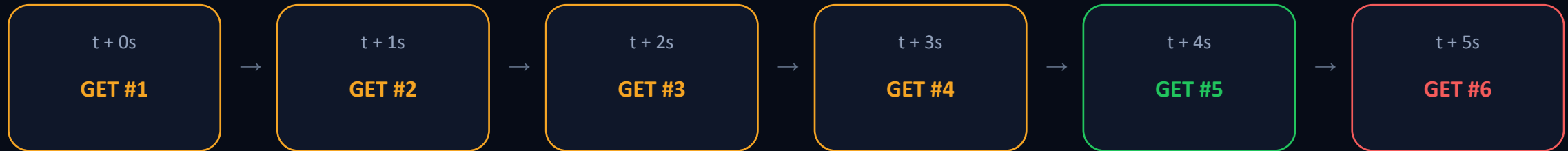
Largest generated output for viewing

Dimensions & aspect-ratio are listed and preserved.

Evidence: storage/variants contains the original name + three outputs; fine until the worker finishes.

Browser short-polling every ~1 second

Begins after 202 · one prompt GET per second · stops at terminal states



- UI updates badge, timeline, timestamps, polling indicator
- Results stay hidden while queued or processing
- AbortController cancels observation on new job or page unload
- Each response returns promptly with current state (short, not long, polling)

When to apply: polling starts after 202 — At processing image clicked — never at file selection.

UI state rendering

Initial

No image, job, results, or polling

Selected

Local preview only — no POST, no job

Uploading

Button disabled, isSubmitting guard

Queued / Processing

Timeline + badge advance per poll

Completed

Polling stops; 3 variants shown

Failed / Retrieval error

Safe message; Try again to resume

Double-click protection: disabled button + isSubmitting guard stop overlapping POSTs.

Retrieval failure $\neq$ processing failure

Situation	Meaning	UI behavior
Status GET fails	Observation failure	“Unable to check status.” Stop polling; offer Try again.
API returns status failed	Processing failure	Safe error; polling stops.
Try again clicked	Resume observation	Poll same status_url; never resubmits the image.

Decision rule: **could not retrieve status = observation failure**; API said failed = processing failure. Never conflate them.

Try again lets completion be discovered even while observation was interrupted.

Single-image numbers

Metric	Formula	Value
Acknowledgement latency	response received – POST start	—
Queue wait	started_at – queued_at	—
Processing duration	completed_at – started_at	—
Job duration	completed_at – queued_at	—
Polling count	# status GET requests	—
Detection delay	client completion time – server completed_at	—

Gathered using script `measure_image.sh`

Five images, one worker — queue becomes visible

Illustrative timeline (fill in real measured durations)

Job A	t=0	claimed immediately
Job B	t=0	queued — waits
Job C	t=0	queued — waits
Job D	t=0	queued — waits
Job E	t=0	queued — waits longest

Interpretation: one worker means later jobs wait in queue — queue wait grows with burst size.

Staggered started_at values prove only one job is processed at a time.

Acceptance checklist

- Initial state has no image, job, results, or polling
- Local selection creates a preview but no server resource
- Overlapping page uploads are prevented
- Handler stores original + queued job, then returns 202
- 202 includes Location, image_id, job_id, status, status_url
- Handler does not generate variants
- One worker records valid state transitions
- Polling after 202: prompt GETs every second
- UI renders queued, processing, completed, failed, retrieval-error
- Polling stops at terminal states and is cancellable
- Try again observes the same job; no resubmission
- Results hidden until every variant is available
- Three variants satisfy dimension / aspect rules
- Single-image and five-image measurements completed
- README lets another person run the app

Why it works, and one limitation

202 vs latency

Acknowledging early lowers response time without removing processing work — the work still happens, just after the response.

One worker queueing

Later jobs wait because one worker is busy; queue wait is real, visible, and expected.

1-second polling

A shorter interval detects completion sooner but issues more requests; one second balances them.

Limitation

Polling is a fixed cadence — a future mechanism (long polling / webhooks / SSE) could push state instead of pulling. Not implemented here.

Scripted 90-second walkthrough

- | | | |
|---|--------------|--|
| 1 | Initial page | Nothing selected, no job, no results, no polling |
| 2 | Select image | Preview appears; Network panel shows NO POST yet |
| 3 | Submit | 202 + status_url; polling begins |
| 4 | Watch | Timeline & badge advance with each 1-second GET |
| 5 | Completed | Polling stops; 3 variants appear |

Docs to bring: DB rows, storage files, Network panel, measurement screen.

ImageLab ships a complete asynchronous loop

Accepted durably → processed by one worker → observed through short polling.

Thank you

Assessment 1 · CMPS 4121